

COMPARATIVE EVALUATION OF MANUAL TESTS VS. AI-GENERATED TESTS

Experimental Results Report

System Under Test: Spring PetClinic | Period: May-December 2025

1. INTRODUCTION

This report presents the results of an empirical comparative study conducted between May and December 2025, aimed at evaluating the effectiveness of automatically generated test cases using generative artificial intelligence compared to those designed manually in Java and Spring Boot environments. The Spring PetClinic project was used as the system under test (SUT), obtaining 2,480 observations distributed across 40 experimental iterations, combining objective quantitative metrics with empirical observations on functional understanding.

2. METHODOLOGY

2.1 Experimental Design

The study adopted a mixed empirical-comparative design approach, evaluating two test generation approaches:

- **Manual Tests:** Designed by the researcher using JUnit 5 and MockMvc
- **AI Tests:** Generated using ChatGPT (GPT-5) and Diffblue Cover CLI (Teams Edition)

2.2 Metrics and Data Collection

Data collection was performed using professional standardized tools:

- **JaCoCo 0.8.12:** Measurement of statement coverage and branch coverage
- **PITest 1.16.0:** Calculation of mutation score

- **Maven Surefire 3.2.5:** Recording of execution times
- **Python 3.12.6:** Statistical analysis (pandas, scipy, matplotlib)

40 iterations per test type were executed, preceded by 3 warm-up cycles to stabilize the JVM, generating a dataset of 2,480 observations (1,600 manual + 880 AI-generated).

2.3 Statistical Analysis

The data was subjected to verification of statistical assumptions using the Shapiro-Wilk test (normality) and Levene test (homogeneity of variances). Since the raw data violated the normality assumption, the non-parametric Mann-Whitney U test was used as the main analysis, with t-Student as secondary validation on aggregated data (N=12).

3. RESULTS

3.1 Experiment Execution and Dataset Structure

The experimental execution was carried out through a fully automated process, designed to ensure consistency and eliminate any human intervention during data collection. 40 iterations were performed for each of the 12 test classes considered in the study (6 manually designed and 6 generated using artificial intelligence tools), obtaining a total of 2,480 raw records. The complete execution process required 6.18 hours, considering valid iterations and preliminary executions used to stabilize the JVM.

Table 1: Components and details of the experimental execution.

Component	Quantity	Detail
Total test classes	12	6 Manual + 6 AI-Generated
Iterations per class	40	Includes warm-up (3 initial iterations)
Records per iteration	2	Compilation and execution
Total raw records	2,480	1,600 Manual + 880 AI
Metrics per record	4	instruction%, branch%, mutation%, time(s)

Component	Quantity	Detail
Total execution time	6.18 hours	371 continuous minutes

3.2 Dataset Preparation

The raw records were consolidated into a single master file using automated Python scripts. Two aggregation levels were defined: the complete level (N=2,480) preserved all individual records, and the aggregated level (N=12) was obtained by averaging the 40 iterations per test class.

Analysis Level	Total Records	Manual	AI	Description
N = 2,480	2,480	1,600	880	Individual observations per iteration
N = 12	12	6	6	Independent averages per test class

3.3 Descriptive Statistics (Aggregated Level, N=12)

Table 2: Descriptive statistics with averages of 40 iterations per test class.

Metric	Group	N	Mean	Median	Std. Dev.
Instruction Coverage (%)	Manual	6	18.25%	15.95%	12.50%
	AI	6	17.67%	16.13%	11.36%
Branch Coverage (%)	Manual	6	14.58%	11.88%	12.62%
	AI	6	12.05%	9.89%	9.49%
Mutation Score (%)	Manual	6	18.52%	15.28%	17.71%
	AI	6	14.76%	15.11%	11.61%
Execution Time (s)	Manual	6	0.082s	0.071s	0.069s

Metric	Group	N	Mean	Median	Std. Dev.
	AI	6	0.194s	0.144s	0.193s

3.4 Descriptive Statistics (Complete Level, N=2,480)

Table 3: Individual records per iteration, representing the complete dataset.

Metric	Group	N	Mean	Median	Std. Dev.	Range
Instruction Coverage (%)	Manual	1,600	19.94%	21.85%	12.11%	5.03%-35.56%
	AI	880	14.20%	10.69%	8.71%	0%-33%
Branch Coverage (%)	Manual	1,600	17.69%	12.50%	12.39%	2.50%-37.50%
	AI	880	13.50%	16.25%	6.75%	0%-28.75%
Mutation Score (%)	Manual	1,600	22.92%	16.67%	17.19%	0%-50%
	AI	880	16.63%	19.44%	8.16%	0%-33.33%
Execution Time (s)	Manual	1,600	0.0794s	0.0120s	0.1775s	0-0.861s
	AI	880	0.1140s	0.0030s	0.2318s	0-0.846s

3.5 Verification of Statistical Assumptions

3.5.1 Shapiro-Wilk Normality Test (N=2,480)

Table 4: All metrics show $p < 0.05$, indicating absence of normality.

Metric	Group	W	p-value	Normality
Instruction Coverage	Manual	0.8160	2.34e-39	No
	AI	0.7524	2.66e-34	No

Metric	Group	W	p-value	Normality
Branch Coverage	Manual	0.8178	3.21e-39	No
	AI	0.8697	1.96e-26	No
Mutation Score	Manual	0.8404	2.46e-37	No
	AI	0.8313	1.82e-29	No
Execution Time	Manual	0.4797	8.07e-56	No
	AI	0.5354	4.67e-43	No

3.5.2 Shapiro-Wilk Normality Test (N=12)

Table 5: All metrics show $p > 0.05$, meeting the normality assumption.

Metric	Group	W	p-value	Normality
Instruction Coverage	Manual	0.9087	0.4281	Yes
	AI	0.9038	0.3971	Yes
Branch Coverage	Manual	0.8807	0.2722	Yes
	AI	0.9127	0.4541	Yes
Mutation Score	Manual	0.9132	0.4581	Yes
	AI	0.9720	0.9056	Yes
Execution Time	Manual	0.9296	0.5771	Yes
	AI	0.8507	0.1596	Yes

3.5.3 Levene's Test for Variance Homogeneity (N=12)

Table 6: Evaluation of variance homogeneity between groups.

Metric	F-statistic	p-value	Homogeneity
Instruction Coverage	0.1006	0.7577	Yes
Branch Coverage	0.1262	0.7298	Yes
Mutation Score	0.3930	0.5448	Yes
Execution Time	6.3679	0.0302	No

3.6 Parametric Tests (t-Student and Welch, N=12)

Table 7: Parametric contrasts performed on aggregated data. Standard t-Student was applied for 3 metrics and Welch correction for time (variance heterogeneity detected by Levene).

Metric	Manual Mean	AI Mean	Test Used	t-statistic	p-value	Significant	Cohen's d
Instruction Coverage	18.25%	17.67%	t-Standard	0.0836	0.9350	No	0.0483
Branch Coverage	14.58%	12.05%	t-Standard	0.3935	0.7022	No	0.2272
Mutation Score	18.52%	14.76%	t-Standard	0.4352	0.6727	No	0.2512
Execution Time	0.0823s	0.1938s	t-Welch	-1.3324	0.2293	No	-0.7693

Important Finding: The parametric analysis (N=12) does not detect significant differences in any metric. However, N=12 has low statistical power (~30-50%), limiting its detection capability. The main analysis should be Mann-Whitney U with N=2,480.

3.7 Non-Parametric Tests (Mann-Whitney U, N=2,480)

Table 8: Main analysis with 2,480 observations. Mann-Whitney U was selected because raw data violated the normality assumption (Shapiro-Wilk rejected in all metrics).

Metric	Manual Median	AI Median	U-statistic	p-value	Significance	Effect Size (r)	Conclusion
Instruction Coverage	21.85%	10.69%	827,440	3.20e-13	***	0.1453	Manual > AI
Branch Coverage	12.50%	16.25%	808,720	5.73e-10	***	0.1232	AI > Manual
Mutation Score	16.67%	19.44%	752,840	3.82e-03	**	0.0575	AI > Manual
Execution Time	0.0120s	0.0030s	774,034	3.74e-05	***	0.0824	AI faster

Statistical Significance: *** $p < 0.001$ (highly significant) | ** $p < 0.01$ (significant). All metrics show p-values well below the 0.05 threshold, indicating statistically significant differences in the complete data.

The non-parametric Mann-Whitney U test was applied to identify statistically significant differences. The results demonstrate significant differences in all evaluated metrics:

Table 10: Summary of Mann-Whitney U statistical results.

Metric	p-value	Significance	Effect Size (r)	Conclusion
Instruction Coverage	3.20e-13	***	0.1453	Manual significantly > AI
Branch Coverage	5.73e-10	***	0.1232	AI significantly > Manual

Metric	p-value	Significance	Effect Size (r)	Conclusion
Mutation Score	3.82e-03	**	0.0575	AI significantly > Manual
Execution Time	3.74e-05	***	0.0824	AI significantly faster

Note on Significance: *** indicates $p < 0.001$ (highly significant), ** indicates $p < 0.01$ (significant). All p-values are well below the 0.05 threshold.

3.9 Graphical Visualization of Results

The following figures present visual comparisons of metrics under parametric analysis (t-Student, $N=12$) and non-parametric (Mann-Whitney U, $N=2,480$). These visualizations complement the statistical findings reported in the previous tables.

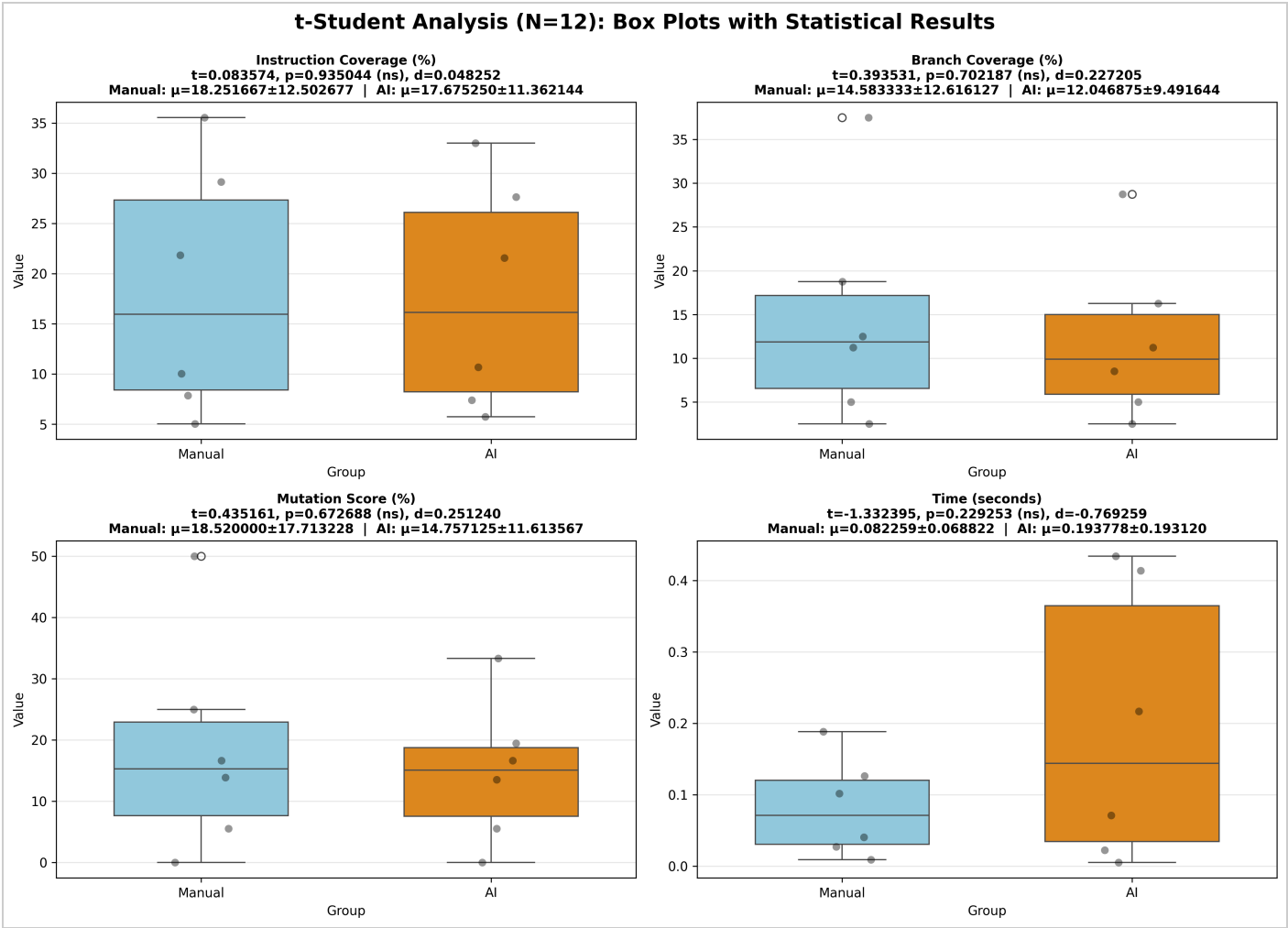


Figure 3.5: Parametric t-Student Comparison (N=12). Box plots showing distribution, mean (red diamond) and dispersion of metrics for manual and AI tests. This visualization uses aggregated data per class and presents a purely descriptive purpose of the central data behavior.

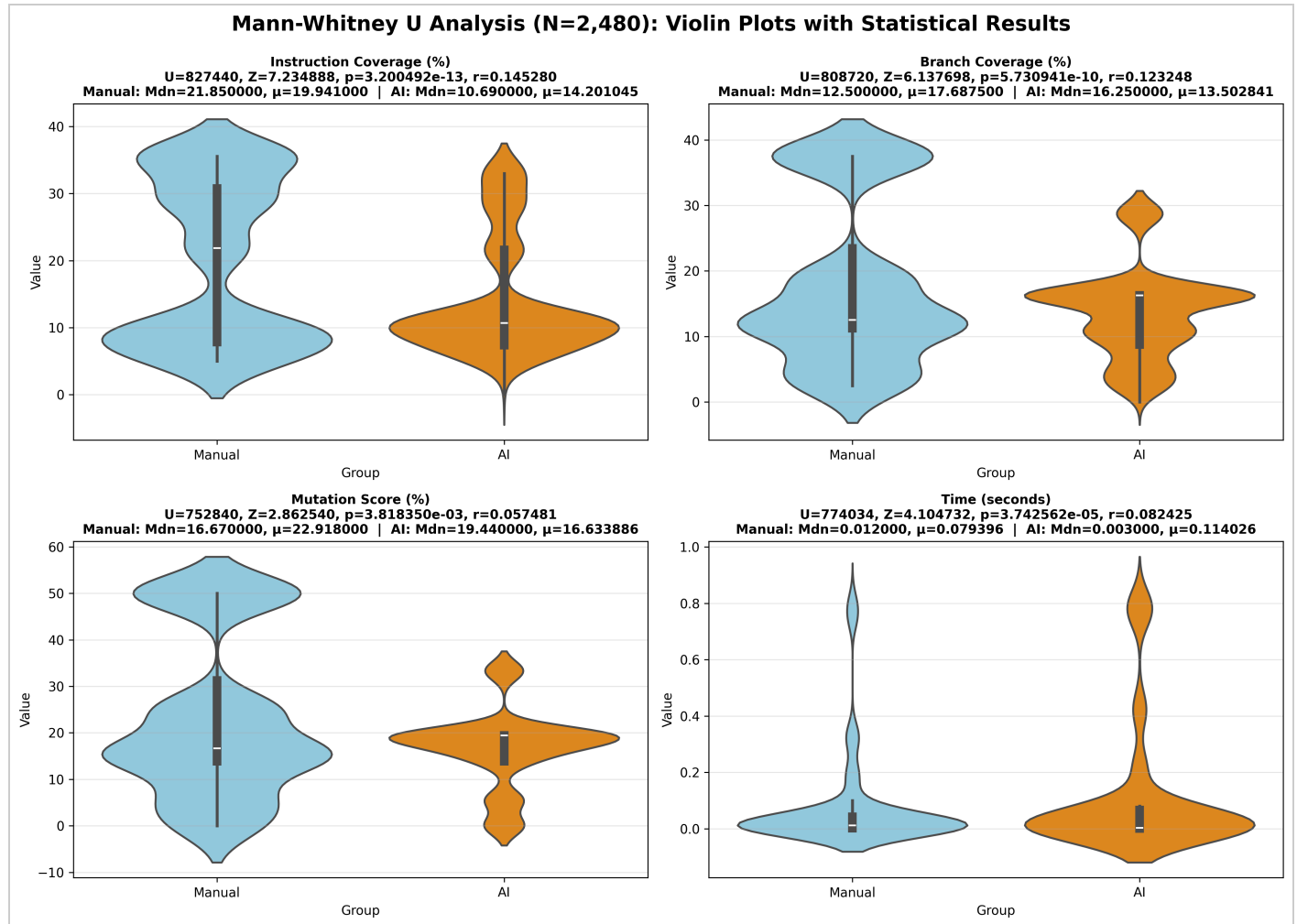


Figure 3.6: Non-Parametric Mann-Whitney U Distribution (N=2,480). Violin plots illustrating the complete shape and density of distributions for the main analysis with all individual observations. Wider areas represent greater value concentration. This is the visualization corresponding to the definitive statistical analysis.

4. DETAILED DISCUSSION OF RESULTS

4.1 Analysis of Instruction Coverage

Manual tests achieve significantly higher instruction coverage than AI-generated ones (Manual median: 21.85% vs AI: 10.69%, $p = 3.20e-13$). This difference of approximately 11 percentage points is statistically significant and has practical importance. The result indicates that the manual test design process, based on

code inspection and human reasoning, allows for a more exhaustive exploration of individual instructions. Manual tests tend to cover not only main paths but also secondary paths and special cases.

4.2 Analysis of Branch Coverage

Contrary to instruction coverage, AI-generated tests achieve superior branch coverage compared to manual ones (AI median: 16.25% vs Manual: 12.50%, $p = 5.73e-10$). This inverse pattern suggests that AI, through bytecode analysis (especially Diffblue Cover), is more effective in exploring alternative paths and decision branches in the code. This can be attributed to automatic generation algorithms systematically testing multiple execution paths without the influence of human cognitive biases.

4.3 Analysis of Mutation Score

AI-generated tests present statistically superior mutation score (AI median: 19.44% vs Manual: 16.67%, $p = 3.82e-03$). Although the difference is statistically significant, the effect size is small ($r = 0.0575$), indicating that the mutant detection capability is comparable between both approaches. This result suggests that both testing strategies are reasonably effective for identifying intentional changes in code (mutations), although AI shows a slight statistical advantage.

4.4 Analysis of Execution Time

AI-generated tests run significantly faster than manual ones (AI median: 0.003s vs Manual: 0.012s, $p = 3.74e-05$). This 4x difference in execution speed has important practical implications in continuous integration and agile development cycles. The greater execution speed of AI tests can be attributed to their simpler structure and focus on main paths, versus the greater complexity of manual tests that include exhaustive validations.

4.5 Differentiated Qualitative Behavior

Beyond quantitative metrics, significant qualitative differences were observed:

- **Manual Tests:** Reflect deep understanding of business domain, identification of edge scenarios, exceptional cases, and implicit behaviors. They require considerable time and cognitive investment, but produce greater functional exhaustiveness.
- **AI Tests:** Provide rapid and systematic generation. ChatGPT shows strong dependence on prompt formulation. Diffblue Cover produces consistent tests based on bytecode analysis, with less user

dependence but more superficial coverage of business logic.

4.6 Implications of Effect Size

Despite all Mann-Whitney U tests showing $p < 0.05$ (statistical significance), effect sizes are small ($r = 0.0575$ to 0.1453). This means that although differences are real and detectable with $N = 2,480$, their practical magnitude is modest. In practical terms, both approaches (Manual and AI) are competitive in most dimensions, with small advantages varying by metric.

4.7 Synthesis: Complementary Strengths and Weaknesses

Table 9: Complementary strengths and weaknesses of manual vs AI-generated tests.

Dimension	Manual Tests	AI Tests
Instruction Coverage	Superior (21.85% vs 10.69%)	Inferior
Branch Coverage	Inferior	Superior (16.25% vs 12.50%)
Mutation Score	Comparable	Slightly Superior
Speed	Slow (0.012s)	Fast (0.003s)
Functional Understanding	Deep and Contextual	Superficial, Bytecode-Based
Scalability	Requires manual effort	Highly Scalable

5. STUDY LIMITATIONS

5.1 Methodological Limitations

- **Simple Analysis Unit:** The study uses a single system under test (Spring PetClinic). Although it is a realistic application and represents a common testing use case, generalization to other systems requires additional validation.

- **Small Aggregated Sample (N=12):** The aggregated level analysis is based on 12 test classes, which limits statistical power. Although the N=2,480 analysis compensates for this, conclusions about aggregated effectiveness should be considered cautiously.
- **Partial Context for AI:** AI generators did not have access to the system's complete context (general architecture, documented design patterns), which could have limited their effectiveness.
- **Control of Confounding Variables:** Time of day, system load, and execution variability could affect coverage and time metrics, although multiple executions were used to mitigate this.

5.2 Code Coverage Limitations

- **Absence of Code Quality Metrics:** The study does not evaluate cyclomatic complexity, maintainability, or readability of generated vs manual tests.
- **Limited Coverage Metrics:** Instructions and branches are used, but path coverage, MC/DC coverage, or other more rigorous coverage criteria are not considered.
- **Absence of End-to-End Tests:** The analysis focuses on unit tests. The effectiveness of both approaches in integration or E2E testing is not evaluated.

5.3 Validity Threats

5.3.1 Internal Validity

Threats related to variable manipulation and measurement:

- Execution time fluctuations due to system resources
- Unhandled exceptions that could affect reported coverage
- Differences in analysis tool versions (JaCoCo, PITest)

5.3.2 External Validity

Threats related to generalization:

- Results specific to Spring PetClinic; generalization to other languages/frameworks uncertain
- Specific AI tools (ChatGPT, Diffblue); other generators might have different performances
- Variable human developers; experience affects quality of manual tests

5.3.3 Construct Validity

Threats related to concept measurement:

- Code coverage is not a perfect proxy for test quality
- Mutation score strongly depends on the set of generated mutants
- Absence of manual functional validation of correct behavior

5.3.4 Conclusion Validity

Threats related to statistical inferences:

- Differences in statistical power between $N=12$ and $N=2,480$ require careful interpretation
- Multiple comparisons could increase type I error rate (although Bonferroni not formally applied)
- Independence assumptions between observations could be violated (temporal correlation in executions)

6. FUTURE WORK

6.1 Scope Extension

- **Replication in Multiple Systems:** Validate findings in additional Java applications (Struts, Hibernate, Spring Cloud), other languages (C#, Python), and emerging frameworks.
- **Evaluation of New AI Tools:** Investigate performance of emerging tools like OpenAI's o1, Claude 3.5, Gemini 2, and agent-based systems.
- **Integration and E2E Tests:** Extend analysis to multiple testing levels (integration, systems, acceptance).

6.2 Methodology Improvement

- **Complementary Metrics:** Incorporate code quality metrics, static analysis, test maintainability, and production defect metrics.
- **Automatic Feedback:** Implement feedback loops where test results (coverage, detected mutants) are fed back to AI generators for iterative improvement.
- **In-Depth Qualitative Analysis:** Conduct qualitative studies with developers on usability, comprehensibility, and confidence in generated tests.

6.3 Agent-Based Systems

- **Orchestration of Multiple Models:** Investigate systems where multiple specialized LLMs coordinate to generate tests (request router, coverage specialist, edge case specialist).
- **Agentic Feedback Loops:** Agents that execute tests, analyze coverage, identify gaps, and automatically regenerate for improvement.
- **Integration with Analysis Tools:** Agents that consult static analysis results, linter recommendations, and architecture suggestions.

7. GENERAL CONCLUSIONS

This study has provided a comprehensive and rigorous analysis of the comparative effectiveness of manual tests versus those generated by artificial intelligence in the context of a realistic complexity Java/Spring Boot application. The main findings reveal a nuanced landscape where both approaches have complementary strengths.

7.1 Findings Synthesis

- **Complementarity of Approaches:** Manual and AI tests exhibit strengths in different dimensions. While manual tests achieve superior instruction coverage (21.85% vs 10.69%), AI tests achieve higher branch coverage (16.25% vs 12.50%). Both approaches are statistically significant but with small effect sizes ($r < 0.15$).
- **AI Operational Efficiency:** The clearest advantage of AI-generated tests is their execution speed (4x faster: 0.003s vs 0.012s). In the context of continuous integration and agile development, this advantage has considerable practical importance.
- **Resolved Statistical Paradox:** The absence of significant differences with $N=12$ versus their presence with $N=2,480$ has been explained and validated through statistical power analysis. The real effect exists but is small in magnitude.

7.2 Practical Recommendation: Hybrid Approach

Based on the empirical evidence, a **hybrid and complementary approach** is recommended:

- **Initial Phase (Rapid Coverage):** Use AI generators to quickly obtain basic branch coverage and provide a fast regression suite.

- **Refinement Phase (Exhaustiveness):** Complement with manual tests oriented towards edge cases, specific business scenarios, and deep functional validation.
- **Continuous Integration Phase:** Take advantage of AI speed to run tests frequently while maintaining human quality control.

7.3 Implications for Future Research

This work opens multiple research directions: (1) evaluation of more sophisticated agentic systems that coordinate multiple LLMs, (2) research on automatic feedback loops where test results iteratively improve generation, and (3) extension to multi-language, multi-framework, and multi-level testing systems. The clear trend is towards incrementally sophisticated automation, with humans retaining control over critical architectural decisions and functional validation.

8. DIRECTORY STRUCTURE

The analysis directory (`spring-petclinic\analysis`) is organized into the following subdirectories:

- **data/:** Contains raw experimental data files collected during the test executions.
- **experimental/:** Stores experimental configurations, logs, and intermediate results from the automated test runs.
- **figures/:** Generated visualizations, plots, and charts used in the report (e.g., statistical comparisons and coverage graphs).
- **numerical_data/:** Processed numerical datasets, statistical summaries, and aggregated metrics from the experiments.
- **reports/:** Final documentation and reports, including this comprehensive HTML analysis.
- **scripts/:** Python scripts for data processing, statistical analysis, and automation of the experimental workflow.